설비 고장을 예측하는 AI 데이터 분석

환경 설정 및 Python 시작

Python 첫 코드와 오류 읽기

posco

설비 고장을 예측하는 AI 데이터 분석

환경 설정 및 Python 시작

설비 데이터와 Python 환경 Part 1

예측 정비와 설비 데이터

Python을 쓰는 이유

개발 환경 3가지

Python 첫 코드와 오류 Part 2

코드 실행과 print()

설비 데이터 출력

오류 메시지 읽기

Python 첫 코드와 오류 읽기

posco

코드가 실행되는 흐름

코드 작성
명령 적기

→

실행
버튼 누르기

→

해석
한 줄씩 읽기

→

출력
화면에 결과

컴퓨터는 적힌 대로 실행

컴퓨터는 코드를 **위에서 아래로 순서대로** 읽음

의도가 아니라 **글자 그대로** 받아들임

그래서 글자 하나, 순서 하나가 결과를 바꿈

코드와 결과의 구분

코드	내가 적는 명령
결과	실행 후 나타남

타이핑은 코드, 나타나면 결과

내가 **타이핑했으면 코드**, 저절로 **나타났으면 결과**

결과 자리에 직접 글자를 쓰는 게 아님

결과를 바꾸려면 **결과가 아니라 코드**를 고침

print 함수란

print는 파이썬에 기본으로 들어 있는 함수 (내장 함수)

괄호 안 내용을 화면에 출력 (영어 print = 찍어내다)

글자도 숫자도 괄호 안에 넣으면 그대로 표시

print는 확인의 도구

print 사용 예시

1. print("안녕하세요") 실행하면 안녕하세요 출력
2. print(75) 실행하면 75 출력

> 데이터가 잘 들어왔는지, 계산이 맞는지 **확인할 때마다** print로 화면에 표시

함수와 괄호의 개념

함수	일해주는 도구
괄호	재료 넣는 입구

지금 필요한 한 문장

함수는 **일해주는 도구**, 괄호는 **재료 넣는 입구**

괄호를 빼먹으면 컴퓨터가 헷갈려 오류 발생

함수의 깊은 내용은 다음 파트에서 다시 배움

print 기본형 입력·실행

목표 - print 기본 형태를 입력하고 실행

한 줄씩 차근차근 손에 익히기

코랩 노트북을 켜고 시작

print 실행하기

새 셀에 입력 후 실행

CODE

```
print("안녕하세요")
print("첫 번째 줄")
print("두 번째 줄")
```

첫 출력 점검

점검

1. print 기본 형태를 따라 썼는가
2. 실행해 결과가 나왔는가
3. print 여러 번이 여러 줄이 되는 것을 봤는가

핵심 - **print 한 번에 한 줄**

숫자 출력하기

숫자는 **따옴표 없이** 괄호 안에 그대로 넣음

온도 print(75), 진동 print(2.3)

소수점이 있는 숫자도 그대로 표시

숫자에 따옴표가 없는 이유

숫자에 따옴표를 안 붙이는 이유

1. 따옴표 없는 숫자는 **계산 가능한 진짜 값**
2. 따옴표를 붙이면 숫자 모양의 글자로 취급

> 지금은 **숫자는 따옴표 없이**만 기억

숫자 출력하기

목표 – 오늘 날짜를 출력하고 직접 바꾸기

오늘 날짜를 숫자로 출력 (예: 2026년 1월 1일 → 260101)

오늘 날짜를 출력한 뒤에 현재 시간까지 보이도록 수정해서 출력하기 (예: 2026년 1월 1일 오전 9시→ 2601010900)

숫자 출력 점검

점검

1. 따옴표 없이 숫자를 출력했는가
2. 숫자를 직접 바꿔 다시 실행했는가
3. 소수점 숫자도 출력해봤는가

따옴표로 문자 출력

글자는 반드시 **따옴표로 감싸서** 출력

큰따옴표든 작은따옴표든 한 쌍이 같으면 됨

따옴표 안은 띄어쓰기·한국어 문장도 그대로

따옴표가 없으면 생기는 일

따옴표를 빼면

1. 따옴표 없는 글자를 어떤 값을 가리키는 이름으로 인식
2. 그런 이름을 만든 적 없어 오류 발생

> 규칙 쌍 - **숫자는 따옴표 없이, 글자는 따옴표로**

숫자와 문자의 차이

구분	작성 방법	컴퓨터 처리
숫자	따옴표 없이	계산 가능
문자	따옴표로 감쌈	글자 그대로

같은 75도 따옴표로 갈림

같은 75라도 따옴표가 있으면 **글자**, 없으면 **숫자**

설비 측정값은 계산해야 하니 따옴표 없이

항목 이름이나 설명은 글자니까 따옴표로

문자 출력하기

목표 - 여러 문자를 따옴표로 출력하고 바꾸기

이름·다짐·설비 이름 무엇이든 가능

따옴표로 감싸는 것만 잊지 않기

문자를 바꿔 실행

기존 문자를 아래 내용으로 바꾸기

CODE

```
print("안녕하세요")
print("저는 설비 데이터를 배웁니다")
# 따옴표를 일부러 빼고 실행해 오류도 관찰
```

문자 출력 점검

점검

1. 문자를 따옴표로 감싸 출력했는가
2. 따옴표 안 내용을 직접 바꿨는가
3. 따옴표를 빼면 오류가 난다는 것을 경험했는가

여러 값 함께 출력

괄호 안에 **쉼표로 나누면** 한 줄에 여러 값

쉼표로 나눈 값 사이엔 자동 띄어쓰기

print("온도", 75) 실행하면 온도 75

이름표와 값을 함께

이름표를 붙이는 출력

1. print(75) 만 보면 무슨 값인지 모름
2. print("온도(℃):", 75) 면 한눈에 파악

항목 이름과 값을 함께 보여주는 형태를 기억

실습 - 측정값 출력 안내

목표 - 온도·진동·압력을 이름표와 함께 출력

글자·숫자·여러 값 출력을 모두 활용

점검 화면처럼 보이게 구성

```
=== 1번 설비 측정값 ===
온도(℃): 75
진동(mm/s): 2.3
```

온도·진동·압력 출력

글자와 숫자 한 번에 print로 출력하는 것 확인

CODE

```
print("=== 1번 설비 측정값 ===")
print("온도(℃):", 75)
print("진동(mm/s):", 2.3)
```

실습 - 측정값 출력 점검

점검

1. 이름표와 값을 쉼표로 함께 출력했는가
2. 값을 직접 바꿔 다시 실행했는가
3. 여러 설비 측정값을 이어서 출력했는가

계산 결과 출력

괄호 안에 **계산식**을 넣으면 계산 후 출력

print(75 + 5) 실행하면 80

컴퓨터를 계산 도구로 쓰기 시작

사칙연산 기호

계산 기호

1. 더하기 + , 빼기 -
2. 곱하기는 별표 * , 나누기는 빗금 /

print(82 - 75) 실행하면 **온도 차이 7** 출력

실습 - 계산 출력

목표 – 임의로 작성한 측정값으로 합·차·평균 계산해 출력

계산기처럼 식을 넣으면 됨

직접 실습 후 값 바꿔 직접 계산

```
두 측정값의 합: 155
온도 변화량: 7
평균 온도: 77.5
```

실습 - 합·차·평균 계산

입력 후 값 바꿔 계산

CODE

```
print("두 측정값의 합:", 75 + 80)
print("온도 변화량:", 82 - 75)
print("평균 온도:", (75 + 80) / 2)
```

실습 - 계산 출력 점검

점검

1. 합과 차를 계산해 출력했는가
2. 괄호로 평균을 올바르게 계산했는가
3. 값을 바꿔 결과가 달라지는 것을 봤는가

주석이란

뒤에 적은 내용은 컴퓨터가 실행하지 않음

오직 사람이 읽는 설명 메모

print(75) # 1번 설비 온도 처럼 사용

주석은 미래의 나를 위한 메모

주석 사용

1. 줄 끝에 # 설명 - 그 줄만 메모 추가
2. 줄 맨 앞에 # - 그 줄 전체가 메모

> 지금은 이해해도 일주일 뒤엔 기억 안 남, **주석이 시간을 아껴줌**

코딩이 빨라지는 단축키

자주 쓰는 단축키 네 가지

1. Tab : 이름 일부만 치고 Tab → 나머지 자동완성 (오타 예방)
2. Ctrl+/ : 선택한 줄을 주석 ↔ 해제 토글 (한 번에)
3. Shift+Enter : 셀 실행 후 다음 셀로
4. Ctrl+Z : 실행 취소 (되돌리기)

> 데이터가 잘 들어왔는지, 계산이 맞는지 **확인할 때마다** print로 화면에 표시

Print는 결과를 확인하는 습관

실행한 뒤에는 결과를 **반드시 눈으로 확인**

컴퓨터는 정확하지만 내가 잘못 시킬 수 있음

결과를 내 예상과 비교하기

결과를 예상과 비교

결과 확인이 중요한 이유

1. 평균이 1000처럼 이상하면 계산 실수 신호
2. 잘못된 결과로 정상이라 판단하면 고장 신호 놓침

실행했으면 결과를 보고 **예상과 비교**하는 습관

print로 중간값 찍어 확인하기

계산이 이상할 때, 중간값을 print로 하나씩 찍어보면 어디서 틀렸는지 바로 보임

CODE

```
print("정상 온도:", 71)
print("현재 온도:", 82)
print("차이 계산 직전:", 82, 71)   # 중간값 확인
print("온도 상승량:", 82 - 71)
```

실습 - 주석 달기

목표 - 만든 코드에 주석을 달아 정리

미래의 나를 위한 메모를 남기기

실습 - 코드에 주석 달기 예시

이전 작성했던 코드에 주석으로 설명을 작성하기

CODE

```
# 1번 설비 측정값 출력
print("온도(℃):", 75)   # 정상 범위
print("진동(mm/s):", 2.3)  # 정상 범위
```

실습 - 주석 정리 점검

점검

1. 줄 끝에 # 주석을 달았는가
2. 줄 맨 앞 한 줄 주석을 만들었는가
3. 주석을 달아도 결과가 안 바뀌는 것을 봤는가

오류는 읽을 정보다

빨간 오류 메시지는 혼내는 게 아니라 **안내문**

어디가 왜 문제인지 단서가 거의 다 들어 있음

읽을 줄 알면 대부분 스스로 해결 가능

오류를 대하는 마음가짐

오류는 실패가 아니라 **과정**

숙련 개발자도 하루에 수십 번 오류를 만남

차이는 안 내는 게 아니라 **빨리 읽고 고치는 것**

오류 메시지 읽는 법

```
print(온도)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_451/1533082916.py in <cell line: 0>()
----> 1 print(온도)

NameError: name '온도' is not defined
```

읽을 요소	메시지 모습	알 수 있는 것
위치	line 1	첫 번째 줄이 문제
종류	NameError	이름 관련 문제
설명	not defined	정의 안 됨

세 가지만 차례로 읽기

메시지가 길어도 볼 것은 **위치·종류·설명** 셋

이 세 가지면 오류의 단서가 거의 다 잡힘

메시지 전체를 다 이해할 필요는 없음

오류 속 영어 단어

자주 나오는 단어

1. Error 오류 , line 줄
2. not defined 정의되지 않음
3. invalid 올바르지 않음 , syntax 문법

조합 예 - **NameError: name ‘temp’ is not defined**

모르면 번역기 OK

영어 공부를 따로 할 필요 없음

같은 단어가 반복되니 곧 익숙해짐

모르면 **번역기**를 써도 됨, 시도가 중요

오류를 스스로 해결하는 습관

혼자 해결하는 4단계

1. 빨간 메시지에서 맨 아랫줄(오류 종류+설명)을 마우스로 복사
2. 검색창에 그대로 붙여넣기 (예: NameError name not defined)
3. 모르는 영어 단어는 번역기에 그대로 붙여넣기
4. 상단 결과 2~3개만 읽고 내 상황과 비슷한 것 적용

데이터가 잘 들어왔는지, 계산이 맞는지 **확인할 때마다** print로 화면에 표시

오류 줄 번호 찾기

메시지의 **line 다음 숫자**가 줄 번호

line 3 이면 코드의 셋째 줄을 살펴보기

화면 왼쪽 줄 번호로 해당 줄 바로 찾기

근처 줄도 함께 확인

줄 번호를 볼 때

1. 알려준 줄이 정확한 위치가 아닐 수 있음
2. 그 줄과 **바로 위아래**를 함께 살피기

그래도 출발점을 알려주니 처음부터 헤매는 것보다 훨씬 빠름

NameError 이해

NameError는 **정의 안 한 이름**을 썼을 때 발생

가장 흔한 원인은 **글자에 따옴표 누락**

print("온도")로 고치면 오류가 사라짐

따옴표 누락이 흔한 원인

NameError를 만나면

1. 내가 따옴표를 빼먹었나 점검
2. 없는 이름을 썼나 점검

> 메시지에 **not defined**가 보이면 이름 문제 신호

실습 - NameError 만들기

목표 - NameError를 만들고 읽고 직접 고치기

통제된 상황에서 오류를 미리 만나보기

진짜 오류 때 침착해지는 연습

실습 - 읽고 고치기

따옴표 빼서 오류, 읽고 고치기

CODE

```
print(NameError 만들기)
# 메시지에서 line, NameError, not defined 찾기
# print("NameError 만들기")로 고쳐 다시 실행
```

실습 - NameError 점검

점검

1. 따옴표를 빼서 NameError를 만들었는가
2. 줄 번호·종류·설명을 찾았는가
3. 따옴표를 넣어 고치고 사라지는 것을 봤는가

SyntaxError 이해

SyntaxError는 **문법이 어긋났을 때** 발생

흔한 원인은 따옴표나 괄호의 **짝이 안 맞음**

보통 invalid syntax 같은 설명이 함께 나옴

짝 맞추기로 해결

SyntaxError 점검

1. print("온도) 처럼 **따옴표 짝**이 안 맞음
2. print("온도" 처럼 **괄호 짝**이 안 맞음

> 여는 게 있으면 닫는 게 있어야 함, **짝 맞추기**가 핵심

실습 - SyntaxError 만들기

목표 - Sy ntax Error를 만들고 짝 맞춰 고치기

따옴표나 괄호를 한쪽만 쓰면 바로 발생

만들고 읽고 고치기 반복

실습 - 짝 맞춰 고치기

짝을 빼서 오류, 읽고 고치기

CODE

```
print("온도)
print("진동"
# SyntaxError와 줄 번호 찾고, 닫는 짝 넣어 고치기
```

실습 - SyntaxError 점검

점검

1. 따옴표 짝을 안 맞춰 오류를 만들었는가
2. 괄호 짝을 안 맞춰 오류를 만들었는가
3. 짝을 맞춰 직접 고쳤는가

오류 대처 4단계

당황 안 하기
과정일 뿐

→

종류·위치 읽기
Error와 line

→

해당 줄 확인
짝·이름 점검

→

고치고 재실행
다시 실행

네 단계를 **하나의 루틴**으로 외워두기

처음 보는 오류도 차근차근 풀 수 있음

오류 해결은 재능이 아니라 **순서를 따르는 일**

실습 - 디버깅 안내

목표 - 여러 오류가 섞인 코드를 하나씩 고치기

오류를 고치는 과정을 디버깅이라 부름

한 번에 하나씩 나타나니 차근차근

실습 - 오류 하나씩 고치기

오류 3개를 차례로 고치기

CODE

```
print(온도, 75)
print("진동", 2.3
print("압력": 4.0)
```

실습 - 디버깅 점검

점검

1. 한 번에 하나씩 오류를 찾아 고쳤는가
2. 종류·위치 읽고 해당 줄 확인했는가
3. 모두 고쳐 정상 출력을 확인했는가

실습 - 점검 리포트 안내

목표 - 제목·측정값·계산·주석을 모은 리포트 출력

온도에는 정상 온도와 같은 메모 주석으로 작성 필수

오늘 배운 출력 기술을 하나로 합쳐 간단한 리포트처럼 완성

```
===== 1번 설비 점검 =====
온도(℃): 82
온도 상승량: 11
```

실습 - 리포트 작성

뼈대 입력 후 값·계산 채우기

CODE

```
print("===== 1번 설비 점검 =====")
print("온도(℃):", 82)   # 정상 71 대비 상승
print("온도 상승량:", 82 - 71)
```

실습 - 리포트 점검

점검

1. 제목 줄을 글자로 출력했는가
2. 측정값을 이름표와 함께 출력했는가
3. 계산 결과를 출력했는가
4. 주석으로 상태 메모를 달았는가

실습 - 리포트 보완 안내

목표 - 체크리스트로 리포트를 점검하고 보완

배운 모든 요소를 빠짐없이 활용했는지 확인

완성도를 스스로 높이기

실습 - 체크리스트 채우기

자기 점검 체크리스트

1. 글자 출력 - 제목·이름표를 따옴표로
2. 숫자 출력 - 측정값을 따옴표 없이
3. 여러 값 - 이름표와 값을 쉼표로
4. 계산 - 상승량·평균 한 줄 이상
5. 주석 - 설명 메모, 결과 확인

감사합니다.

posco